

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

ЗВІТ

про виконання лабораторної роботи № 5
з курсу “Життєвий цикл програмного забезпечення. Управління ІТ
проектами”
на тему: “Версіонування програмного забезпечення. Локальна
система контролю версій”

Виконав:

Студент. гр. ФеП-41 Фіняк Олег

Перевірив:

Франів В.А.

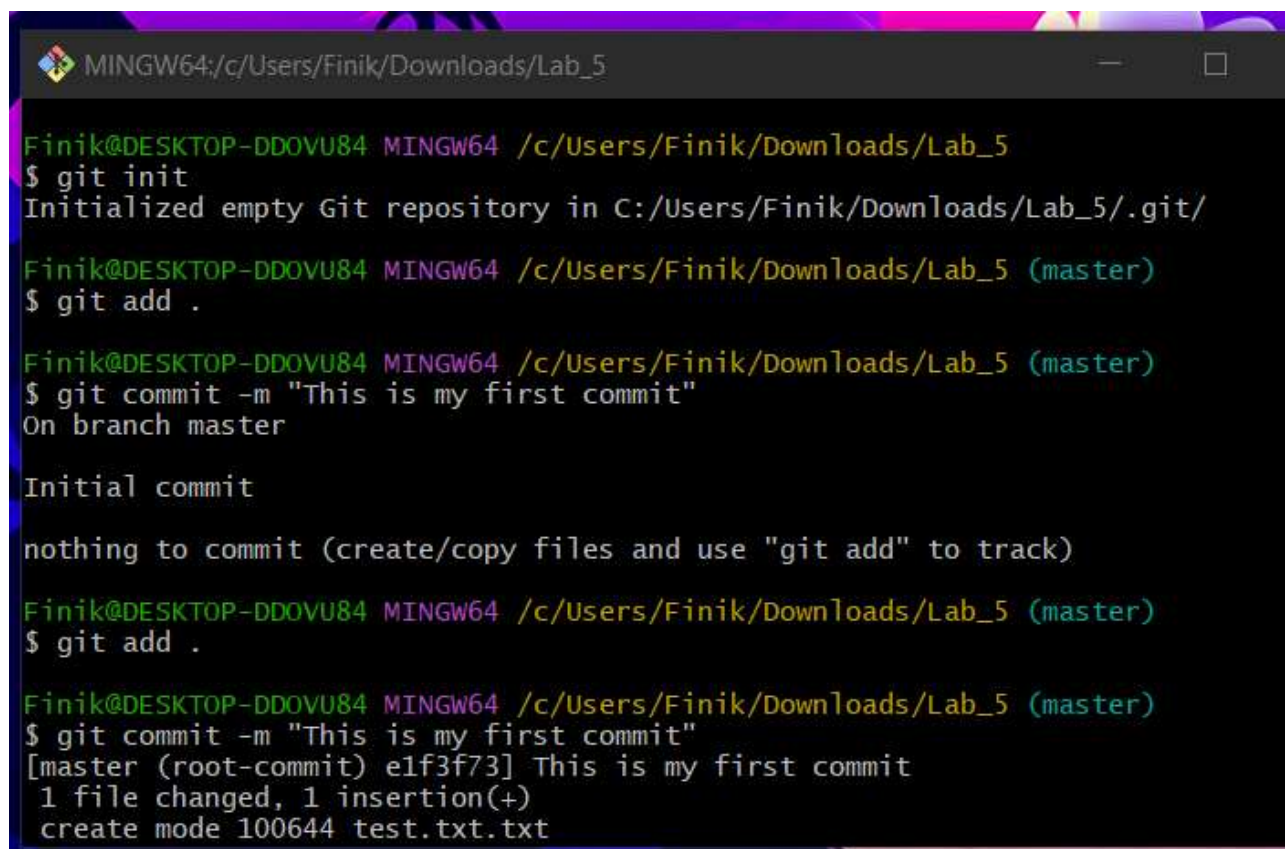
Мета роботи: Мета лабораторної роботи полягає у вивченні концепції версіонування програмного забезпечення, розумінні важливості та переваг використання систем контролю версій, а також у освоєнні практичних навичок створення локальних репозиторіїв та ефективного користування ними базуючись на технології Git.

Вимоги до результатів виконання лабораторної роботи:

1. Створити локальний репозиторій з проектом з попередніх лабораторних робіт.
2. Описати ключові стадії версіонування проекту.
3. Проаналізувати плюси та мінуси використання системи контролю версій GIT.

Порядок виконання роботи:

1. Освоїти вище наведений теоретичний матеріал.
2. Встановити систему контролю версій Git.
3. Використовуючи наявний проект з попередньої лабораторної роботи ініціалізувати для цього проекту локальний git репозиторій.
4. Створити декілька нових файлів та додати їх до вашого git репозиторію.
5. Створити перший коміт для добавлених файлів з повідомленням: This is my first commit.

A screenshot of a terminal window titled 'MINGW64:/c/Users/Finik/Downloads/Lab_5'. The terminal shows the following sequence of commands and output:

```
Finik@DESKTOP-DDOVU84 MINGW64 /c/Users/Finik/Downloads/Lab_5
$ git init
Initialized empty Git repository in C:/Users/Finik/Downloads/Lab_5/.git/

Finik@DESKTOP-DDOVU84 MINGW64 /c/Users/Finik/Downloads/Lab_5 (master)
$ git add .

Finik@DESKTOP-DDOVU84 MINGW64 /c/Users/Finik/Downloads/Lab_5 (master)
$ git commit -m "This is my first commit"
On branch master

Initial commit

nothing to commit (create/copy files and use "git add" to track)

Finik@DESKTOP-DDOVU84 MINGW64 /c/Users/Finik/Downloads/Lab_5 (master)
$ git add .

Finik@DESKTOP-DDOVU84 MINGW64 /c/Users/Finik/Downloads/Lab_5 (master)
$ git commit -m "This is my first commit"
[master (root-commit) elf3f73] This is my first commit
1 file changed, 1 insertion(+)
create mode 100644 test.txt.txt
```

6. Здійснити зміни у файлах та зробити другий коміт.

```
Finik@DESKTOP-DDOVU84 MINGW64 /c/Users/Finik/Downloads/Lab_5 (master)
$ git add .
pick 06b6a12 # Added basic logic
# This is a combination of 2 commits.
```

7. Об'єднати два попередні коміти в один.

Для злиття двох попередніх комітів в один було використано інтерактивний rebase. `git rebase -i HEAD~2`

Ми змінили в другому коміті pick на squash

```
MINGW64:/c/Users/Finik/Downloads/Lab_5
pick 06b6a12 # Added basic logic
squash 04fe9b5 # some features

# Rebase e1f3f73..04fe9b5 onto e1f3f73 (2 commands)
#
# Commands:
# p, pick <commit> = use commit
# r, reword <commit> = use commit, but edit the commit message
# e, edit <commit> = use commit, but stop for amending
# s, squash <commit> = use commit, but meld into previous commit
# f, fixup [-C | -c] <commit> = like "squash" but keep only the previous
#                               commit's log message, unless -C is used, in which case
#                               keep only this commit's message; -c is same as -C but
#                               opens the editor
# x, exec <command> = run command (the rest of the line) using shell
# b, break = stop here (continue rebase later with 'git rebase --continue')
# d, drop <commit> = remove commit
# l, label <label> = label current HEAD with a name
# t, reset <label> = reset HEAD to a label
# m, merge [-C <commit> | -c <commit>] <label> [# <oneline>]
#     create a merge commit using the original merge commit's
#     message (or the oneline, if no original merge commit was
.git/rebase-merge/git-rebase-todo[+] [unix] (12:51 03/05/2026) 1,6 Top
-- INSERT --
```

```
[detached HEAD d70c3df] combine
Date: Sun May 3 12:48:58 2026 +0300
1 file changed, 4 insertions(+)
Successfully rebased and updated refs/heads/master.
```

Висновок:

Використання системи контролю версій Git є критично важливим для розробки програмного забезпечення.

Плюси: Git забезпечує повний локальний контроль над історією проекту. Це дозволяє легко відстежувати, які саме зміни були внесені у код, швидко скасовувати невдалі рішення та відновлювати втрачені дані. Система є дивовижно швидкою та простою в архітектурі, оскільки всі операції (створення знімків, індексація) виконуються локально на комп'ютері розробника.

Мінуси: Робота виключно з локальним репозиторієм створює єдину точку відмови (якщо жорсткий диск пошкодиться, історія комітів може бути втрачена). Тому локальний Git максимально розкриває свій потенціал лише у поєднанні з віддаленими децентралізованими сервісами (наприклад, GitHub).

Література:

1. Scott Chacon, Ben Straub. Pro Git. Apress. 2014. 456 p.
2. Jon Loeliger, Matthew McCullough. Version Control with Git. O'Reilly Media. 2012. 456 p.
3. Ryan Hodson. Git Essentials. Packt Publishing. 2015. 194 p.
4. Roger Dudler. Git Pocket Guide. O'Reilly Media. 2013. 234 p.
5. Seth Robertson, Steve Oualline. Practical Git: For Everyday Use. No Starch Press. 2014. 300 p.